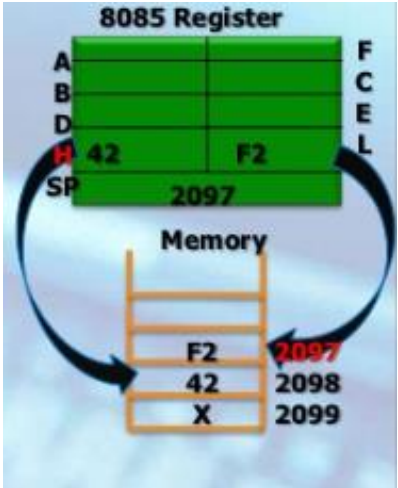
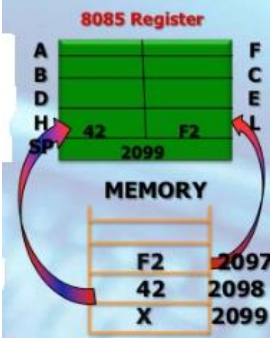


Sahrdaya College of Engineering and Technology, Kodakara		
Answer Key		
APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY FIFTH SEMESTER B.TECH DEGREE EXAMINATION, DECEMBER 2021		
Course Code: EET303		
Course Name: MICROPROCESSORS AND MICROCONTROLLERS		
PART A		
		<i>Answer all questions. Each question carries 3 marks</i>
	Q1	List the control and status signals of 8085
	Ans	- These signals are used to identify the nature of operation. There are 3 control signal and 3 status signals. Three control signals are $\overline{RD}$, $\overline{WR}$ & ALE. $\overline{RD}$ – It is a read control signal. When the signal on this pin is low, selected memory on input output device is read and the information is placed on the data bus. $\overline{WR}$ – This signal indicates that the data on the data bus is to be written into a selected memory or IO location. - It is a write control signal. When the signal is low, the data on the data bus is written to the memory or input output device. - ALE – It is a positive going pulse generated when a new operation is started by the microprocessor. When the pulse goes high, it indicates address. When the pulse goes down it indicates data. Three status signals are IO/M, S0 & S1. IO/M - This signal is used to differentiate between IO and Memory operations, i.e. when it is high indicates IO operation and when it is low then it indicates memory operation. S1 & S0 - These signals are used to identify the type of current operation.
	Q2	Explain the operation of the following 8085 instructions: (i) DAD D (ii) INR M (iii) XCHG.
	Ans	(i) DAD D : This instruction will add the contents of DE register pair, 1020H to the contents of HL register pair, 2050H. It will store the result, 3070H in the HL register pair. (ii) INR M – Increment the contents of memory, the address of which is in HL pair (iii) XCHG – This instruction exchanges the contents of the register H with that of D and of L with that of E Eg. DE = 2040H, HL 7080H XCHG – This instruction will load the data into registers as follows

		H = 20H, L = 40H, D = 70H and E = 80H
	Q3	Define PUSH and POP instructions of 8085.
	Ans	PUSH - The stack pointer is decremented by one to 2098H, and the contents of the H register are copied to memory location 2098H. - The stack pointer register is again decremented by one to 2097H, and the contents of the L register are copied to memory location 2097H. - The contents of the register pair HL are not destroyed; however HL is made available for delay counter.  POP H - The contents of the top of the stack location shown by the stack pointer are copied in the L register and the stack pointer register is incremented by one to 2098 H - The contents of the top of the stack(now it is 2098H) are copied in the H register, and the stack pointer is incremented by one.  - During pushing, the stack operates in a ‘decrement then store’ style.

		• The stack pointer is decremented first, then the information is placed on the stack. • During popping, the stack operates in a ‘use then increment’ style • The information is retrieved from the top of the stack and then the pointer is incremented. • The SP pointer always points to ‘the top of the stack’.								
Q4	Calculate the loop delay if the register pair BC of 8085 microprocessor is loaded with 8000H. Assume the system clock frequency to be 3.072 MHz.									
Ans	256 millisecond									
Q5	Explain maskable and non-maskable interrupts of 8085.									
Ans	Maskable Interrupt • Maskable interrupts are those interrupts that can be delayed or denied. Any interrupt which can be disabled by giving appropriate instructions is termed as maskable interrupts. Non-Maskable Interrupts • These interrupts cannot be disabled using any instructions. • The <i>TRAP</i> interrupt is one of the non-maskable interrupts.									
Q6	Distinguish between hard and soft real-time systems.									
	<table><tr><th><u>HARD RTS</u></th><th><u>SOFT RTS</u></th></tr><tr><td>1. In a hard or immediate real-time system, the completion of an operation after its deadline is considered useless - ultimately, this may lead to a critical failure of the complete system.</td><td>1. A soft real-time system on the other hand will tolerate such lateness, and may respond with decreased service quality.</td></tr><tr><td>2. Hard real-time systems are typically found interacting at a low level with physical hardware, in embedded systems.</td><td>2. Soft real-time systems are typically those used where there is some issue of concurrent access and the need to keep a number of connected systems up to date with changing situations.</td></tr><tr><td>3. Hard real-time systems are used when it is imperative that an event is reacted to within a strict deadline.</td><td>3. These can operate to a latency of seconds.</td></tr></table>	<u>HARD RTS</u>	<u>SOFT RTS</u>	1. In a hard or immediate real-time system, the completion of an operation after its deadline is considered useless - ultimately, this may lead to a critical failure of the complete system.	1. A soft real-time system on the other hand will tolerate such lateness, and may respond with decreased service quality.	2. Hard real-time systems are typically found interacting at a low level with physical hardware, in embedded systems.	2. Soft real-time systems are typically those used where there is some issue of concurrent access and the need to keep a number of connected systems up to date with changing situations.	3. Hard real-time systems are used when it is imperative that an event is reacted to within a strict deadline.	3. These can operate to a latency of seconds.	
<u>HARD RTS</u>	<u>SOFT RTS</u>									
1. In a hard or immediate real-time system, the completion of an operation after its deadline is considered useless - ultimately, this may lead to a critical failure of the complete system.	1. A soft real-time system on the other hand will tolerate such lateness, and may respond with decreased service quality.									
2. Hard real-time systems are typically found interacting at a low level with physical hardware, in embedded systems.	2. Soft real-time systems are typically those used where there is some issue of concurrent access and the need to keep a number of connected systems up to date with changing situations.									
3. Hard real-time systems are used when it is imperative that an event is reacted to within a strict deadline.	3. These can operate to a latency of seconds.									

		4. Missing a deadline is a total system failure. 5. The goal of a hard real-time system is to ensure that all deadlines are met. 6. The preemption period for the hard real time task in worst case should be less than a few micro sec. Example: a car engine control system, medical systems such as heart pacemakers and industrial process controllers.	4. The usefulness of a result degrades after its deadline, thereby degrading the system's quality of service. 5. The goal becomes meeting a certain subset of deadlines in order to optimize some application specific criteria. 6. The preemption period for the soft real time task in worst case may be about a few ms. Example: the software that maintains and updates the flight plans for commercial airliners, Live audio-video systems
Q7		Explain the operation of the following 8051 instructions: (i) MOV A,@R1 (ii) CPL A (iii) SWAP A.	
Ans		(i) Mov A, @R1 This instruction copies the contents of memory location whose address is specified in the register R1 from selected register bank to accumulator (or) copy the contents of the address in R1 to accumulator. (ii) CPL A - Complement Accumulator (iii) SWAP A – Swap the upper nibble and lower nibble of A.	
Q8		Define the assembler directives of 8051.	
Ans		The following are some more widely used directives of the 8051 ORG [Origin]: - The ORG directive is used to indicate the beginning of the address. - The number that comes after ORG can be either in HEX or in decimal. - If the number is not followed by 'H', it is decimal and the assembler will convert it into hex. For Example ORG 0100H - Some assemblers use ".ORG" [notice the dot] instead of "ORG" for the origin directive. For that you need to check your assembler. EQU [Equate]: - EQU is used to define a constant without occupying a memory location. - Here uses EQU for the counter constant and then the constant is used to load the R4 register. COUNT EQU 22 MOV R4,#COUNT After executing the instruction "MOV R4,#COUNT", the register R4 will be loaded with the value 22 [# sign indicates it is a value, notice it].	

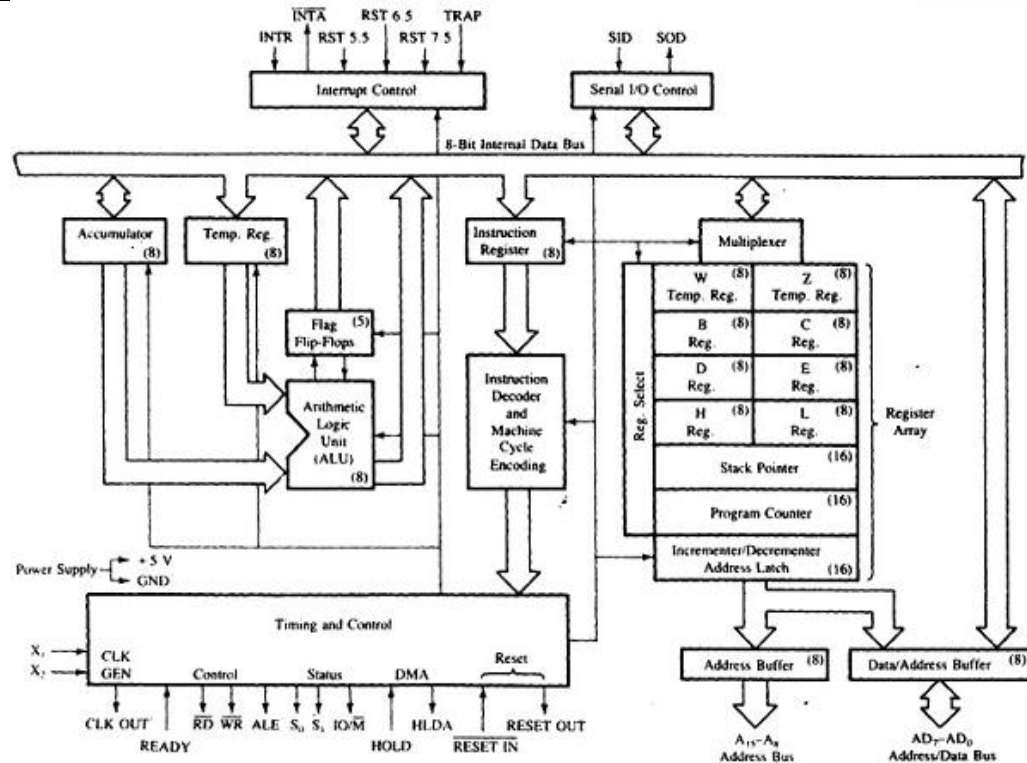
	END Directive: - this indicates to the assembler the end of the source (asm)file - END is the last line of 8051 program - After the END, anything written will be ignored by the assembler																																																
Q9	List the interrupts of 8051.																																																
	8051 has 5 interrupt signals, i.e. INT0, TFO, INT1, TF1, RI/TI. Each interrupt can be enabled or disabled by setting bits of the IE register and the whole interrupt system can be disabled by clearing the EA bit of the same register. - Timer 0 overflow interrupt - TF0 - External hardware interrupt - INT0 - Timer 1 overflow interrupt - TF1 - External hardware interrupt - INT1 - Serial communication interrupt - RI/TI																																																
Q10	Explain the TMOD register of 8051.																																																
	(MSB) (LSB) <table><tr><td>GATE</td><td>C/T</td><td>M1</td><td>M0</td><td>GATE</td><td>C/T</td><td>M1</td><td>M0</td></tr><tr><td colspan="4">Timer 1</td><td colspan="4">Timer 0</td></tr></table> GATE Gating control when set. The timer/counter is enabled only while the INTx pin is high and the TRx control pin is set. When cleared, the timer is enabled whenever the TRx control bit is set. C/T Timer or counter selected cleared for timer operation (input from internal system clock). Set for counter operation (input from Tx input pin). M1 Mode bit 1 M0 Mode bit 0 <table><tr><th>M1</th><th>M0</th><th>Mode</th><th>Operating Mode</th></tr><tr><td>0</td><td>0</td><td>0</td><td>13-bit timer mode</td></tr><tr><td>0</td><td>1</td><td>1</td><td>8-bit timer/counter THx with TLx as 5-bit prescaler</td></tr><tr><td>0</td><td>1</td><td>1</td><td>16-bit timer mode</td></tr><tr><td>1</td><td>0</td><td>2</td><td>16-bit timer/counters THx and TLx are cascaded; there is no prescaler</td></tr><tr><td>1</td><td>0</td><td>2</td><td>8-bit auto reload</td></tr><tr><td>1</td><td>0</td><td>2</td><td>8-bit auto reload timer/counter; THx holds a value that is to be reloaded into TLx each time it overflows.</td></tr><tr><td>1</td><td>1</td><td>3</td><td>Split timer mode</td></tr></table>	GATE	C/T	M1	M0	GATE	C/T	M1	M0	Timer 1				Timer 0				M1	M0	Mode	Operating Mode	0	0	0	13-bit timer mode	0	1	1	8-bit timer/counter THx with TLx as 5-bit prescaler	0	1	1	16-bit timer mode	1	0	2	16-bit timer/counters THx and TLx are cascaded; there is no prescaler	1	0	2	8-bit auto reload	1	0	2	8-bit auto reload timer/counter; THx holds a value that is to be reloaded into TLx each time it overflows.	1	1	3	Split timer mode
GATE	C/T	M1	M0	GATE	C/T	M1	M0																																										
Timer 1				Timer 0																																													
M1	M0	Mode	Operating Mode																																														
0	0	0	13-bit timer mode																																														
0	1	1	8-bit timer/counter THx with TLx as 5-bit prescaler																																														
0	1	1	16-bit timer mode																																														
1	0	2	16-bit timer/counters THx and TLx are cascaded; there is no prescaler																																														
1	0	2	8-bit auto reload																																														
1	0	2	8-bit auto reload timer/counter; THx holds a value that is to be reloaded into TLx each time it overflows.																																														
1	1	3	Split timer mode																																														
PART B <i>Answer any one full question from each module. Each question carries 14 marks</i>																																																	

Module 1

Q11(a)

Explain the architecture of 8085 microprocessor with the help of a neat functional block diagram

Ans



Accumulator

The accumulator is an 8 bit register that is part of the Arithmetic /logic unit (ALU). This register is used to store 8 bit data and to perform arithmetic and logical operations. The result of an operation is stored in the accumulator. The accumulator is also identified as register A. The accumulator is used for data transfer between an I/O port and memory location.

Registers

- General Purpose Registers(B, C, D,E,H,L) (BC pair, DE pair, HL pair)
- Temporary register(W and Z)
- Special purpose registers(Accumulator, Flag, Instruction register)
- 16 bit registers (Program counter, Stack pointer)

Register sets

- The 8085 microprocessor has six general-purpose registers to store 8-bit data; these are identified as B, C, D, E, H and L. They can be combined as register pair like BC, DE and HL – to perform some 16-bit operations.
- The programmer can use these registers to store or copy data into the registers by using data copy instructions.

	Temporary register Two additional registers. W & Z are included in the register array. These register are used to hold 8 bit data during the execution of some instructions. They are not available to the programmer. Program counter It is a 16 bit registers, which is used to store the memory address location of the next instruction to be executed. Stack Pointer (SP): It is a 16 bit registers which acts like a stack i.e., this is used to arrange the data neatly. It is incremented or decremented by 2 during PUSH and POP operation Flag register - It is an 8 bit register. It consists of 5 flip flops which changes its status according to the result stored in an accumulator. It is also known as status register. It is connected to the ALU. Instruction register and decoder - Instruction register holds the instruction that is currently being processed. Timing and control unit It provides timing and control signal to the microprocessor to perform operations. Following are the timing and control signals, which control external and internal circuits – Control Signals: READY, RD', WR', ALE Status Signals: S0, S1, IO/M' DMA Signals: HOLD, HLDA RESET Signals: RESET IN, RESET OUT Serial input /Output control It controls the serial data communication by using these two instructions. SID (Serial input data and SOD (Serial output data) - SID(serial input data): This is used for transferring of data into the memory serially. - SOD(serial output data): This is used for transferring of data from memory to external device
Q11(b)	Identify the machine cycles of the following instructions: (i) MOV B, M (ii) MVI A, 05H (iii) STA 2065H (iv) OUT 01H
Ans	(i) MOV B,M 2 Machine cycle – Opcode Fetch + Memory Read (ii) MVI A, 05H 2 Machine cycle – Opcode Fetch + Memory Read (iii) STA 2065H 4 Machine cycle - Opcode Fetch+ Memory Read+Memory Read+MW (iv) OUT 01H

	3 Machine cycle - OF+MR+IOW
Q12(a)	Sketch and explain the timing diagram of the instruction MOV M, A with opcode 77H stored in the memory location 4500H.
Ans	The timing diagram illustrates the execution of the MOV M, A instruction. It is divided into two main phases: Opcode fetch (T₁ to T₄) and Memory write (T₁₁ to T₁₃). The clock (CLK) is shown as a periodic square wave. The address bus (AD₀-AD₇) carries the opcode 00H in T₁, 77H in T₂, and 00H in T₁₁, 77H in T₁₂. The data bus (A₈-A₁₅) carries the memory address 45H in T₃ and 46H in T₁₃. The ALE signal is active high during T₁ and T₁₁. The RD signal is active low during T₂-T₄ and T₁₂-T₁₃. The WR signal is active low during T₁₁-T₁₃.
Q12(b)	Explain the addressing modes of 8085 with suitable examples.
Ans	- The way by which the microprocessor identifies the operands for a particular instruction is known as Addressing mode. - Addressing modes in 8085 is classified into 5 groups - Immediate addressing mode - Direct addressing mode - Register addressing mode - Register indirect addressing mode - Implicit addressing mode Immediate Addressing Mode - In this type of addressing mode the operand is specified in the instruction itself. The data will be part of the program instruction - Consider this instruction:

	ADI 34H – This instruction adds the immediate data 34H to the accumulator. Direct Addressing Mode - In this mode of addressing, the address of the data (operand) is specified within the instruction. - In immediate addressing mode the data itself is specified within instruction, but in direct addressing mode the address of the data is specified in the instruction. Example: OUT 10H LDA 4100H STA 2000H Consider the instruction STA 2000H - When this instruction is executed, the contents of the accumulator are stored in the memory location specified. In the above example the contents of accumulator are stored in memory location 2000H. Register Addressing Mode - In this mode, the data is copied from one register to another. For example: MOV A, B: means data in register B is copied to register A. Register Indirect Addressing Mode - This is indirect way of addressing. In this mode the instruction specifies the name of the register in which the address of the data is available. - In this mode, the data is transferred from one register to another by using the address pointed by the register. Example - MOV A, M - This instruction will move the contents of memory location, whose address is in H-L register pair to the accumulator Implicit Addressing Mode - This mode doesn't require any operand; the data is specified by the opcode itself. Example: CMA complements the contents of accumulator.
--	--

Module 2

Q13(a)	Ten bytes of data are stored in consecutive memory locations starting at 2050H. With suitable explanations, write an assembly language program in 8085 to transfer the entire block of data to new memory locations starting at 2070H.																													
Ans	Input <table border="1"><tr><td>2050</td><td>2051</td><td>2052</td><td>2053</td><td>2054</td><td>2055</td><td>2056</td><td>2057</td><td>2058</td><td>2059</td></tr><tr><td>01</td><td>02</td><td>03</td><td>04</td><td>05</td><td>06</td><td>07</td><td>08</td><td>09</td><td>10</td></tr></table>										2050	2051	2052	2053	2054	2055	2056	2057	2058	2059	01	02	03	04	05	06	07	08	09	10
2050	2051	2052	2053	2054	2055	2056	2057	2058	2059																					
01	02	03	04	05	06	07	08	09	10																					

		Output																				
		<table><tr><td>2070</td><td>2071</td><td>2072</td><td>2073</td><td>2074</td><td>2075</td><td>2076</td><td>2077</td><td>2078</td><td>2079</td></tr><tr><td>01</td><td>02</td><td>03</td><td>04</td><td>05</td><td>06</td><td>07</td><td>08</td><td>09</td><td>10</td></tr></table>	2070	2071	2072	2073	2074	2075	2076	2077	2078	2079	01	02	03	04	05	06	07	08	09	10
2070	2071	2072	2073	2074	2075	2076	2077	2078	2079													
01	02	03	04	05	06	07	08	09	10													
		Programs																				
		MVI C, 0AH LXI H, 2050H LXI D, 2070H Loop MOV A,M STAX D INX H INX D DCR C JNZ Loop HLT																				
Q13(b)		The contents of accumulator and C register are 87H and 79H respectively. Find the contents of the accumulator and flags after the execution of the instruction ADD C																				
Ans		Accumulator - 87H 1 0 0 0 0 1 1 1 C Register - 79H 0 1 1 1 1 0 0 1 ----- 1 0 0 0 0 0 0 0 ----- Carry flag = 1 Sign flag = 0 Zero flag = 1																				
Q14(a)		Write and explain an assembly language program in 8085 to perform BCD to Binary conversion.																				
Ans		Sample Problem: (2200H) = 67H (2300H) = 6 x 0AH + 7 = 3CH + 7 = 43H Source Program: LDA 2200H : Get the BCD number MOV B, A : Save it ANI 0FH : Mask most significant four bits MOV C, A : Save unpacked BCDI in C register MOV A, B : Get BCD again ANI 0FH : Mask least significant four bits RRC : Convert most significant four bits into unpacked BCD2 RRC RRC RRC MOV B, A : Save unpacked BCD2 in B register XRA A : Clear accumulator (sum = 0) MVI D, 0AH : Set D as a multiplier of 10 Loop: ADD D : Add 10 until (B) = 0 DCR B : Decrement BCD2 by one JNZ Loop : Is multiplication complete? i if not, go back and add again																				

		ADD C : Add BCD1 STA 2300H : Store the result HLT : Terminate program execution
14(b)		Explain the significance of stack memory while executing CALL and RET instructions in 8085.
		Call addr 16 • It is unconditional CALL used to call a subroutine program. When this instruction is executed, the address of the next instruction in the program counter is pushed to the stack. • The 16 – bit address (which is the address of the subroutine program) given in the instruction is loaded in the program counter. Now, the processor will start executing the instructions stored in this call address. • Three byte instruction • Immediate addressing • Five machine cycles (Opcode fetch (6T), MR(3T), MR(3T), MW(3T), MW (3T) = 18T RET • RET – Return to the main program • It is an unconditional return instruction. • This instruction is placed at the end of the subroutine program, in order to return to the main program. When this instruction is executed, the top of the stack is popped to (loaded in) the program counter. • 1 byte instruction • Register indirect addressing • Three machine cycles (Opcode fetch (4T), MR(3T), MR(3T)) = 10T
Module 3		
15(a)		Explain the interfacing of seven segment LED with 8085

Ans

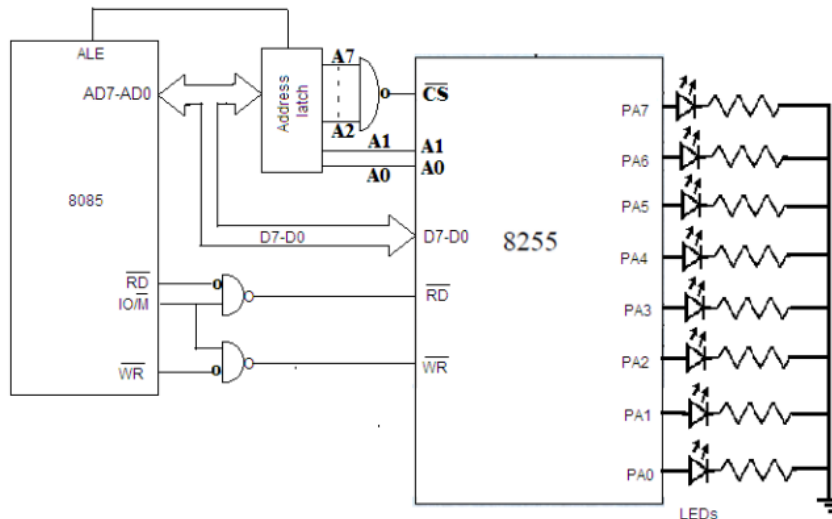


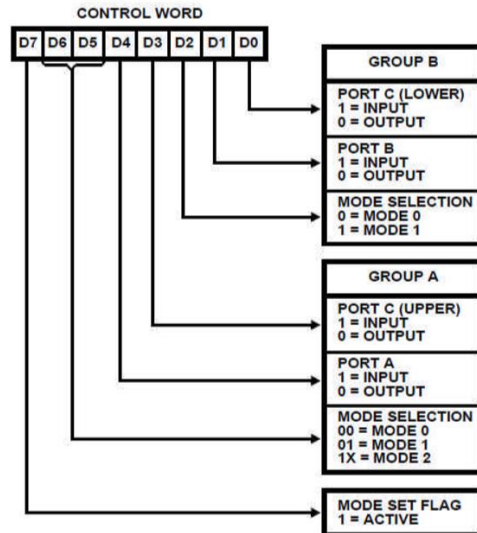
Figure: Interfacing LEDs with 8085

- 8085 microprocessor is interfaced with input devices(DIP switches) and output devices (LED, LCD) using 8255 PPI.
- The figure shows the interfacing of DIP switches and LEDs with microprocessor.
- 8255 PPI is interfaced using address lines, data lines and control line RD and WR
- LEDs are connected to any one of the port by configuring the port in simple IO mode
- An interfacing diagram with 8 LEDs is shown below
- LEDs are connected to the ground through current limiting resistors
- When logic 1 is send to any pin of Port A, the LED will conduct the current and it will glow
- When logic 0 is send to any pin of Port A, then the LEDs will be off.

15(b)

Draw the control word format for the I/O mode of 8255. Hence frame the control word for the following configuration in Mode 0 operation:
Ports A, B and CL - output ports, Port CU- input port.

Ans



Control word Configuration

Ports A, B and CL - output ports, Port CU- input port.

CONTROL WORD

D7	D6	D5	D4	D3	D2	D1	D0
1	0	0	0	1	0	0	0

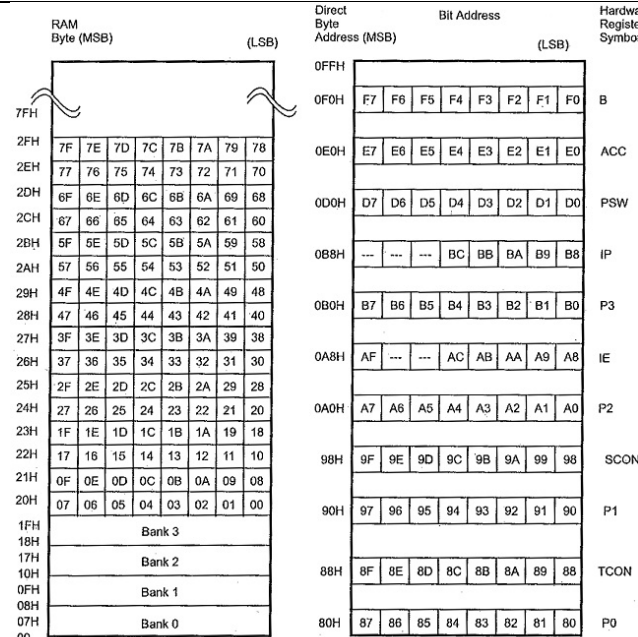
1 0 0 0 1 0 0 0 = 88H

Control word = 88H

16(a)

With the help of a neat sketch, explain the register organization and SFR of 8051.

Ans



- The 8051 has 4K bytes of internal program memory(ROM) and 256 bytes of internal data memory(RAM).
- The Program Memory of the 8051 Microcontroller is used for storing the program to be executed i.e. instructions. The Data Memory on the other hand, is used for storing temporary variable data and intermediate results.

Data Memory (RAM) of 8051 Microcontroller

- The 8051's on-chip memory consists of 256 memory bytes organized as follows:

First 128 bytes: 00h to 1Fh Register Banks
 20h to 2Fh Bit Addressable RAM
 30 to 7Fh General Purpose RAM

Next 128 bytes: 80h to FFh Special Function Registers

Internal RAM Organization

- It Consist of totally 256 RAM memory locations ; The lower 128 memory locations are called General Purpose Area. The internal RAM(this 128 memory locations) of 8051 is organized into three distinct areas.

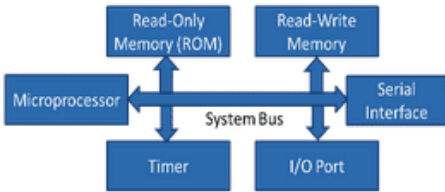
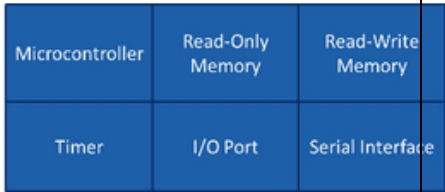
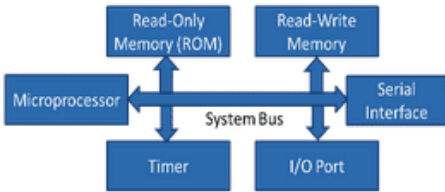
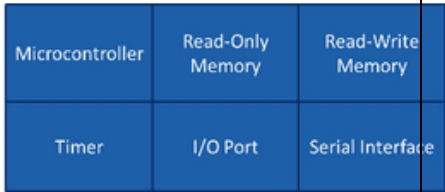
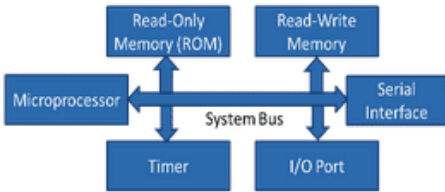
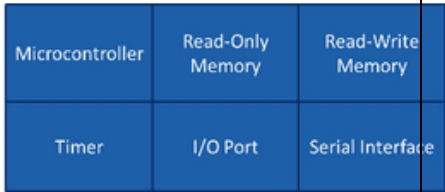
1. Register Bank
2. Bit addressable
3. General Purpose RAM

- A total of 32 bytes from location 00 to 1F hex are set aside for register banks and the stack.
- A total of 16 bytes from locations 20H to 2FH are set aside for bit – addressable read/write memory
- A total of 80 bytes from locations 30H to 7FH are used for read and write storage, or what is normally called a scratch pad. These 80 locations of RAM are widely used for the purpose of storing data and parameters by 8051 programmers
- The location 08H to 7FH can also be used as LIFO STACK. The upper 128 memory locations are called as SPECIAL PURPOSE AREA and consist of registers associated with ALU , on-chip MEMORIES and PERIPHERALS. The type of ON-CHIP RAM is SRAM [or Static RAM]

8051 register banks (working Registers)

- A total of 32 bytes of RAM are set aside for the register banks and stack.
- These 32 bytes are divide into 4 banks of registers in which each bank has 8 registers, R0 – R7.
- RAM locations from 0 to 7 are set aside for bank 0 of R0 - R7 where R0 is RAM location 0, R1 is RAM location 1, R2 is location 2, and so on, until memory location 7, which belongs to R7 of bank 0.
- The second bank of registers R0 to R7 starts at RAM location 08 and goes to location 0FH.
- The third bank of R0 – R7 starts at memory location 10H and goes to location 17H.

	- Finally, RAM locations 18H to 1FH are set aside for allocated into 4 banks. Bit – addressable RAM - Of the 128 – byte RAM of the 8051, only 16 bytes are bit addressable. The rest must be accessed in byte format. The bit – addressable RAM locations are 20H to 2FH. - These 16 bytes provide 128 bits of RAM bit - addressability since $16 \times 8 = 128$. - They are addressed as 0 to 127 (in decimal) or 00 to 7FH. Therefore, the bit addresses 0 to 7 are for the first byte of internal RAM location 20H, and 8 to 0FH are the bit addresses of the second byte of RAM location 21H, and so on. - The last byte of 2FH has bit addresses of 78H to 7FH. In the figure the internal RAM locations 20 – 2FH are both byte – addressable and bit addressable. General Purpose RAM - The RAM area above bit addressable area from 30H to 7FH is called general purpose RAM. - It is addressable as byte. - These 80 bytes of Internal RAM memory are available for general-purpose data storage. - The only problem is that we have only 4 banks and very often the task of bank switching and keeping track of register bank usage is tedious and prone to errors, - For this reason in many applications we use RAM locations 30 7FH as scratch pad and leave addresses 8 – 1FH for stack usage. - That means that we use R0 – R7 of bank 0, and if we need more registers we simply use RAM locations 30 – 7FH. - The general purpose RAM can be accessed using direct or indirect addressing modes. - Examples of direct and indirect addressing: MOV A, 6Ah ; reads contents of address 6Ah to accumulator Special Function Registers (SFRs) - The group of registers, used to perform special functions and are located immediately above the 128 bytes of RAM are called special function registers. - The second 128 bytes are used to store Special Function Registers (SFR) that 8051 controller program can configure and control the ports, timer, interrupts, serial communication, and other tasks. - The SFR is the upper area of addressable memory, from address 0x80 to 0xFF. This area consists of a series of memory-mapped ports and registers. - Out of these 128 Memory Locations (80H to FFH), there are only 21 locations that are actually assigned to SFRs. Each SFR has one Byte Address and also a unique name which specifies its purpose.
--	---

	All port input and output can therefore be performed by get and set operations on SFR port name such as P3.																										
16(b)	Differentiate between microprocessors and microcontrollers.																										
Ans	<table border="1"> <thead> <tr> <th>Microprocessor</th><th>Microcontroller</th></tr> </thead> <tbody> <tr> <td>  </td><td>  </td></tr> <tr> <td>Microprocessor is heart of Computer system</td><td>Microcontroller is a heart of Embedded system</td></tr> <tr> <td>It is just a processor, Memory and I/O components have to be connected externally</td><td>Microcontroller has external processor along with internal memory and I/O components</td></tr> <tr> <td>Since memory and I/O has to be connected externally, the circuit becomes large.</td><td>Since memory and I/O are present internally, the circuit is small.</td></tr> <tr> <td>Cannot be used in compact systems and hence inefficient</td><td>Can be used in compact systems and hence it is an efficient technique</td></tr> <tr> <td>Cost of the entire system increases</td><td>Cost of the entire system is low</td></tr> <tr> <td>Due to external components, the entire power consumption is high. Hence it is not suitable to used with devices running on stored power like batteries</td><td>Since external components are low, total power consumption is less and can be used with devices running on stored power like batteries</td></tr> <tr> <td>Most of the Microprocessors do not have power saving features.</td><td>Most of the micro controllers have power saving modes like idle mode and power saving mode. This helps to reduce power consumption even further.</td></tr> <tr> <td>Since Memory and I/O components are all external, each instruction will need external operation, hence it is relatively slower.</td><td>Since components are internal, most of the operations are internal instruction, hence speed is fast.</td></tr> <tr> <td>Microprocessors have less number of registers, hence more operations are memory based.</td><td>Microprocessors have more number of registers, hence the programs are easier to write</td></tr> <tr> <td>Microprocessors are based on Von Neumann model/architecture where program and data are stored in same memory module.</td><td>Microcontrollers are based on Harvard architecture where program and data memory are separate.</td></tr> <tr> <td>Mainly used in personal computers</td><td>Used mainly in washing machine, MP3 players</td></tr> </tbody> </table>	Microprocessor	Microcontroller			Microprocessor is heart of Computer system	Microcontroller is a heart of Embedded system	It is just a processor, Memory and I/O components have to be connected externally	Microcontroller has external processor along with internal memory and I/O components	Since memory and I/O has to be connected externally, the circuit becomes large.	Since memory and I/O are present internally, the circuit is small.	Cannot be used in compact systems and hence inefficient	Can be used in compact systems and hence it is an efficient technique	Cost of the entire system increases	Cost of the entire system is low	Due to external components, the entire power consumption is high. Hence it is not suitable to used with devices running on stored power like batteries	Since external components are low, total power consumption is less and can be used with devices running on stored power like batteries	Most of the Microprocessors do not have power saving features.	Most of the micro controllers have power saving modes like idle mode and power saving mode. This helps to reduce power consumption even further.	Since Memory and I/O components are all external, each instruction will need external operation, hence it is relatively slower.	Since components are internal, most of the operations are internal instruction, hence speed is fast.	Microprocessors have less number of registers, hence more operations are memory based.	Microprocessors have more number of registers, hence the programs are easier to write	Microprocessors are based on Von Neumann model/architecture where program and data are stored in same memory module.	Microcontrollers are based on Harvard architecture where program and data memory are separate.	Mainly used in personal computers	Used mainly in washing machine, MP3 players
Microprocessor	Microcontroller																										
																											
Microprocessor is heart of Computer system	Microcontroller is a heart of Embedded system																										
It is just a processor, Memory and I/O components have to be connected externally	Microcontroller has external processor along with internal memory and I/O components																										
Since memory and I/O has to be connected externally, the circuit becomes large.	Since memory and I/O are present internally, the circuit is small.																										
Cannot be used in compact systems and hence inefficient	Can be used in compact systems and hence it is an efficient technique																										
Cost of the entire system increases	Cost of the entire system is low																										
Due to external components, the entire power consumption is high. Hence it is not suitable to used with devices running on stored power like batteries	Since external components are low, total power consumption is less and can be used with devices running on stored power like batteries																										
Most of the Microprocessors do not have power saving features.	Most of the micro controllers have power saving modes like idle mode and power saving mode. This helps to reduce power consumption even further.																										
Since Memory and I/O components are all external, each instruction will need external operation, hence it is relatively slower.	Since components are internal, most of the operations are internal instruction, hence speed is fast.																										
Microprocessors have less number of registers, hence more operations are memory based.	Microprocessors have more number of registers, hence the programs are easier to write																										
Microprocessors are based on Von Neumann model/architecture where program and data are stored in same memory module.	Microcontrollers are based on Harvard architecture where program and data memory are separate.																										
Mainly used in personal computers	Used mainly in washing machine, MP3 players																										
Module 4																											
17(a)	Classify 8051 instructions into various categories. Give examples for each.																										
Ans	The 8051 instructions can be divided into the following six groups:																										

	1. Data transfer instructions 2. Arithmetic instructions 3. Logical instructions 4. Branch instructions 5. Subroutine instructions 6. Bit manipulation instructions Data transfer Instruction set • Data transfer instructions are used to transfer data between an internal RAM location and SFR location without going through the accumulator. • Data can also be transferred between the internal and external RAM by using indirect addressing. • Example 1. Mov A, Rn Example: Mov A, R0 This instruction copies the contents of the register R0 of selected register bank to the accumulator. 2. Mov A, direct Example: MOV A, 30H This instruction copies the contents of memory location whose address is 30H to the accumulator. Arithmetic Operations • The arithmetic operations of 8051 include increment, decrement, addition, subtraction, multiplication, division and decimal operations. Incrementing and Decrementing 1. ADD A, Rn Eg. ADD A, R2 – Add contents of A and R2 and store result in A 2. ADD A, direct Eg. ADD A, 20H - Adds contents of A and memory whose address is 20H and store result in A 3. ADD A, @Ri Eg. ADD A, @R2 - Adds contents of A and memory whose address is given by register R2 and store result in A LOGICAL INSTRUCTIONS ANL Destination, Source ANL performs the bitwise logical- AND operation between the variables indicated and stores the result in the destination variable. No flags are affected. 1. ANL A, Rn Ex. ANL A, R2 - Logically ANDs A and R2 and store result in A 2. ANL A, direct Ex. ANL A, 20H
--	--

	Logically ANDs contents of A and memory location whose address is 20H and stores result in A Branch Instruction - Branch instruction group consists of jump and call instructions. - Jump and CALL instructions change the flow of the program by changing the contents of program counter. - A jump permanently changes the program flow whereas call temporarily changes the program flow to allow another part of the program to run. - There are 3 types of jump instructions. They are:- - Relative jump (Range : (+127 to -127)Decimal, (+7FH to - 80H) - Short Absolute jump : (Range 0000H to 07FFH) - Long Absolute jump : (Range 0000H to FFFFH) Bit manipulation Instruction 8051 has 128 bit addressable memory, bit addressable SFRs and bit addressable PORT pins. - It is possible to perform following bit wise operations for these bit addressable locations. - CLR C - Clear carry flag - CLR bit - clear direct bit The indicated bit is cleared (reset to zero). No other flags are affected. CLR can operate on the carry flag or any directly addressable bit.
17(b)	Explain the addressing modes of 8051 with suitable examples.
	- The method by which the operand is specified in an instruction is called addressing modes. 8051 addressing modes are classified as follows. - Immediate addressing. - Register addressing. - Direct addressing. - Register Indirect addressing. - Indexed addressing - Implied addressing 1. Immediate addressing modes In this addressing mode the data is provided as a part of instruction itself. Example - Mov A, #0AFH In these instructions, the # symbol is used for immediate data In this instruction, the immediate data is AFH, but one 0 is added at the beginning. So when the data is starting with A to F, the data should be preceded by 0. 2. Register addressing

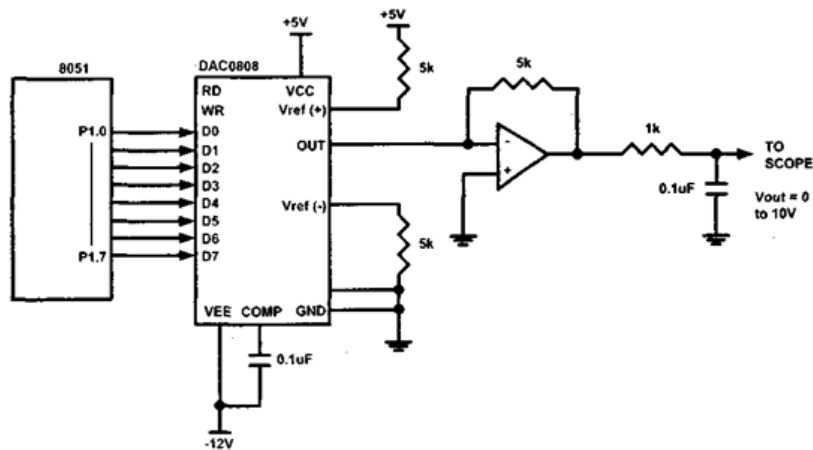
		In register addressing mode, the instruction will specify the name of the register in which the data is available. Example 1. Mov A, R2 Content of R2 register is copied into Accumulator 3. Direct Addressing mode In direct addressing mode, the address of the data is directly specified in the instruction. The direct address can be the address of an internal data RAM location (00H to 7FH) or the address of a special function register (80H to FFH) Example MOV A, 07H The address of the R7 register of bank is 07. This instruction will move the content of the R7 register to the Accumulator. 4. Indirect or Register Indirect addressing mode - In this mode, the instruction specifies the name of the register in which the address of the data is available - The Internal data RAM locations (00H to 7FH) can be addressed indirectly through registers R1 and R0. The external RAM can be addressed indirectly through DPTR. Example MOV A, @R0 - In the instructions, the @ symbol is used for register indirect addressing. 5. Indexed addressing mode - Only program memory can be accessed in the index addressing. Either the DPTR or PC can be used as an index register. The destination operand is always the register A. Example MOV A, @ A+DPTR - This instruction copies the contents of memory locations pointed by sum of the accumulator A and the register DPTR into the accumulator A. 6. Implied addressing mode - In implied addressing mode, the instruction itself specifies the data to be operated by the instruction Example CPL C Complement Carry flag
18(a)		Define data types in C and explain
Ans		Data types A good understanding of C data types for 8051 can help programmers to create smaller hex files Unsigned char

	• Signed char • Unsigned int • Signed int • Sbit (single bit) • Bit and sfr Unsigned char • Since the 8051 is an 8 bit microcontroller, the character data type is the most natural choice for many applications. • The unsigned char is an 8-bit data type that takes a value in the range of 0 – 255 (00 – FFH). • In situations where + and – are needed to represent a given quantity such as temperature, the use of the signed char data is a must. • unsigned char z; Signed char • The signed char is an 8 bit data type that uses the most significant bit (D7 of D7 – D0) to represent the – or + value. • As a result, we have only 7 bits for the magnitude of the signed number, giving us values from -128 to +127. Unsigned int • The unsigned int is a 16-bit data type that takes a value in the range of 0 to 65535 (0000 – FFFFH) • In the 8051, unsigned int is used to define 16-bit variables such as memory addresses . • It is also used to set counter values of more than 256. • Since the 8051 is an 8 bit microcontroller and the int data type takes two bytes of RAM, we must not use the int data type unless we have to. • Since registers and memory accesses are in 8-bit chunks, the misuse of int variables will result in a larger hex file • Such misuse is not a big deal in PCs with 256 megabytes of memory, 32 bit Pentium registers and memory accesses, and a bus speed of 133MHz. Signed int
--	---

	- Signed int is a 16 bit data type that uses the most significant bit (D15 – D0) to represent the – or + value. - As a result, we have only 15 bits for the magnitude of the number, or values from -32, 768 to +32, 767 Sbit (Single bit) - The sbit keyword is a widely used 8051 C data type designed specifically to access single – bit addressable registers. - It allows access to the single bits of the SFR registers. - Some of the SFRs are bit – addressable. - Among the SFRs that are widely used and are also bit, addressable are ports P0 – P3. - Eg: sbit MYBIT = P1^0 Bit and sfr - The bit data type allows access to single bits of bit – addressable memory spaces 20 – 2FH. - Notice that while the sbit data type is used for bit – addressable SFRs, the bit data type used for the bit – addressable section of RAM space 20 – 2FH.
18(b)	Write an 8051 C program to toggle all the bits of P0 and P2 continuously with a 250ms delay.
	<pre>#include <reg51.h> Void MSDelay (unsigned int); Void main(void) { while (1) { P0 = 0*55; P2 = 0*55; MSDelay (250); P0 = 0*AA; P2 = 0*AA; MSDelay (250); } }</pre>

	<pre>void MSDelay (unsigned bit itime) { unsigned int i, j; for (i=0; i<itime;i++ for (j=0; j<1275; j++; }</pre>								
Module 5									
19(a)	Explain SCON and SBUF registers in 8051.								
	SCON <table><tr><td>SM0</td><td>SM1</td><td>SM2</td><td>REN</td><td>TB8</td><td>RB8</td><td>TI</td><td>RI</td></tr></table> SM0 SCON.7 Serial port mode specifier SM1 SCON.6 Serial port mode specifier SM2 SCON.5 Used for multiprocessor communication. (Make it 0.) REN SCON.4 Set/cleared by software to enable/disable reception. TB8 SCON.3 Not widely used. RB8 SCON.2 Not widely used. TI SCON.1 Transmit interrupt flag. Set by hardware at the beginning of the stop bit in mode 1. Must be cleared by software. RI SCON.0 Receive interrupt flag. Set by hardware halfway through the stop bit time in mode 1. Must be cleared by software. <i>Note:</i> Make SM2, TB8, and RB8 = 0. SBUF register(Serial Buffer) • SBUF is an 8-bit register used solely for serial communication in the 8051. • For a byte of data to be transferred via the TxD line, it must be placed in the SBUF register. • Similarly, SBUF holds the byte of data when it is received by the 8051's RxD line. SBUF can be accessed like any other register in the 8051. • Example <pre>MOV SBUF, #'D' ; Load SBUF = 44H, ASCII for 'D' MOV SBUF, A ; copy accumulator into SBUF MOV A, SBUF ; copy SBUF into accumulator</pre> The moment a byte is written into SBUF, it is framed with the start and stop bits and transferred serially via the TxD pin. Similarly, when the bits are received serially via RxD, the 8051 deframes it by eliminating the stop and start bits, making a byte out of the data received, and then placing it in the SBUF.	SM0	SM1	SM2	REN	TB8	RB8	TI	RI
SM0	SM1	SM2	REN	TB8	RB8	TI	RI		

19(b)	Explain Interrupt Enable and Interrupt Priority registers of 8051																																			
	Interrupt Enable Register D7D0 <table><tr><td>EA</td><td>--</td><td>ET2</td><td>ES</td><td>ET1</td><td>EX1</td><td>ET0</td><td>EX0</td></tr></table> EAIE.7 Disables all interrupts. If EA = 0, no interrupt is acknowledged. If EA = 1, each interrupt source is individually enabled or disabled by setting or clearing its enable bit. --IE.6 Not implemented, reserved for future use.* ET2IE.5 Enables or disables Timer 2 overflow or capture interrupt (8052 only). ESIE.4 Enables or disables the serial port interrupt. ET1IE.3 Enables or disables Timer 1 overflow interrupt. EX1IE.2 Enables or disables external interrupt 1. ET0IE.1 Enables or disables Timer 0 overflow interrupt. EX0IE.0 Enables or disables external interrupt 0. *User software should not write 1s to reserved bits. These bits may be used in future flash microcontrollers to invoke new features. Interrupt Priority Register (MSB)(LSB) <table><tr><td></td><td>IP.7</td><td>IP.6</td><td>IP.5</td><td>IP.4</td><td>IP.3</td><td>IP.2</td><td>IP.1</td><td>IP.0</td></tr><tr><td>Direct address B8H</td><td>--</td><td>--</td><td>PT2</td><td>PS</td><td>PT1</td><td>PX1</td><td>PT0</td><td>PX0</td></tr><tr><td>Bit address</td><td>BF</td><td>BE</td><td>BD</td><td>BC</td><td>BB</td><td>BA</td><td>B9</td><td>B8</td></tr></table> Clear for giving low priority for external interrupt 1 (INT1) Set for giving high priority for external interrupt 1 (INT1) Clear for giving low priority for external interrupt 0 (INT0) Set for giving high priority for external interrupt 0 (INT0)	EA	--	ET2	ES	ET1	EX1	ET0	EX0		IP.7	IP.6	IP.5	IP.4	IP.3	IP.2	IP.1	IP.0	Direct address B8H	--	--	PT2	PS	PT1	PX1	PT0	PX0	Bit address	BF	BE	BD	BC	BB	BA	B9	B8
EA	--	ET2	ES	ET1	EX1	ET0	EX0																													
	IP.7	IP.6	IP.5	IP.4	IP.3	IP.2	IP.1	IP.0																												
Direct address B8H	--	--	PT2	PS	PT1	PX1	PT0	PX0																												
Bit address	BF	BE	BD	BC	BB	BA	B9	B8																												
20(a)	Design a DAC interfacing circuit with 8051																																			



MC1408 DAC (or DAC0808)

In the MC1408 (DAC0808), the digital inputs are converted to current (I_{out}), and by connecting a resistor to the I_{out} pin, we convert the result to voltage. The total current provided by the I_{out} pin is a function of the binary numbers at the DO – D7 inputs of the DAC0808 and the reference current (I_{ref}), and is as follows:

$$I_{out} = I_{ref} \left(\frac{D7}{2} + \frac{D6}{4} + \frac{D5}{8} + \frac{D4}{16} + \frac{D3}{32} + \frac{D2}{64} + \frac{D1}{128} + \frac{D0}{256} \right)$$

where DO is the LSB, D7 is the MSB for the inputs, and I_{ref} is the input current that must be applied to pin 14. The I_{ref} current is generally set to 2.0 mA. Figure 13-18 shows the generation of current reference (setting $I_{ref} = 2$ mA) by using the standard 5-V power supply and 1K and 1.5K-ohm standard resistors. Some DACs also use the zener diode (LM336), which overcomes any fluctuation associated

20(b)

Write an 8051 program to transfer letter “A” serially at 4800 baud rate

```

MOV    TMOD,#20H    ;Timer 1, mode 2(auto-reload)
MOV    TH1,#-6      ;4800 baud rate
MOV    SCON,#50H    ;8-bit, 1 stop, REN enabled
SETB   TR1          ;start Timer 1
AGAIN: MOV    SBUF,#"A" ;letter "A" to be transferred
HERE:  JNB    TI,HERE ;wait for the last bit
CLR    TI           ;clear TI for next char
SJMP   AGAIN        ;keep sending A

```